

A Algorithm for Generating a CRPQ's Canonical Logical Plan

Algorithm 3: Get a CRPQ's Canonical Logical Plan

Input: CRPQ $Q(\bar{x}) = \bigwedge_{i=1}^k R_i(y_i, z_i)$
Output: Canonical CRPQ logical plan $T_{Q(\bar{x})}$

```

1 Function GetCRPQCanonicalPlan( $Q(\bar{x})$ ):
2   for  $1 \leq i \leq k$  do
3      $T_{R_i} \leftarrow \text{GetRPQCanonicalPlan}(R_i(y_i, z_i))$ 
4    $\text{planned} \leftarrow \{R_1\}$ 
5    $T \leftarrow T_{R_1}$ 
6   while  $|\text{planned}| < k$  do
7     for  $2 \leq i \leq k$  do
8       if  $R_i \notin \text{planned}$  then
9         if  $y_i \in V \vee y_i \in \overline{x_T}$  then
10           $T \leftarrow T \overset{\overline{x_T} \hat{\oplus} \langle z_i \rangle, d_i}{\cdot} (T, T_{R_i})$ 
11          else if  $z_i \in V \vee z_i \in \overline{x_T}$  then
12             $T' \leftarrow \rho_{y_i/\text{tmp}} \circ \rho_{z_i/y_i} \circ \rho_{\text{tmp}/z_i} (\text{INV}^{(y_i, z_i)}(T_{R_i}))$ 
13             $T \leftarrow T \overset{\overline{x_T} \hat{\oplus} \langle y_i \rangle, d_i}{\cdot} (T, T')$ 
14   return  $T$ 

15 Function GetRPQCanonicalPlan( $R(y, z)$ ):
16   if  $R = S?$  then
17      $T \leftarrow \text{GetRPQCanonicalPlan}(S(y, z))$ 
18     return  $\text{UNION}^{(y, z), d}(T, \langle V_{G_T}, \{\langle v, v \rangle_I \mid v \in V_{G_T}\} \rangle)$ 
19   else if  $R = S^*$  then
20      $T \leftarrow \text{GetRPQCanonicalPlan}(S(y, z))$ 
21     return  $KC^{(y, z), d}(T)$ 
22   else if  $R = S^+$  then
23      $T \leftarrow \text{GetRPQCanonicalPlan}(S(y, z))$ 
24     return  $KC^{(y, z), d}(T)$ 
25   else if  $R = S_1/S_2/\dots/S_m$  then
26      $R' \leftarrow \bigwedge_{i=1}^m S_i(x_i, x_{i+1})$ , where  $y = x_1, z = x_{m+1}$ 
27     return  $\text{GetCRPQCanonicalPlan}(R'(y, z))$ 
28   else if  $R = S_1|S_2|\dots|S_m$  then
29     for  $1 \leq i \leq m$  do
30        $T_i \leftarrow \text{GetRPQCanonicalPlan}(S_i(y, z))$ 
31      $T \leftarrow T_1$ 
32     for  $2 \leq i \leq m$  do
33        $T \leftarrow \text{UNION}^{(y, z), d}(T, T_i)$ 
34   return  $T$ 

```

could be preserved if these operators are redefined to assign rank values as hyperedge properties.

- Aggregation is not composable because it outputs scalar values, serving as a terminal operation rather than producing a hypergraph.

Furthermore, the hypergraph query model facilitates graph analytical tasks that operate on entire graphs, such as PageRank, community detection, and all-pairs shortest path. Because our queries output a hypergraph rather than a tabular result set, requiring no intermediate translation steps such as CONSTRUCT in SPARQL or defining graph views in SQL/PGQ. The design of new algorithms tailored specifically to these hypergraph result structures is a possible future research direction.

B Expressivity of the Proposed Algebra for Query Constructs Beyond CRPQs

Query constructs beyond CRPQs, such as aggregation, filters, and ranking are crucial for mature graph processing systems. The algebra proposed in this paper can fully express CRPQs and effectively handles the foundational graph matching phase. Richer constructs are then executed as subsequent operations on the CRPQ results.

Under the proposed hypergraph model, operations traditionally executed on a row-by-row basis treat a hyperedge as the fundamental computation unit. The composability of these constructs within our algebraic framework varies depending on their semantics:

- Filters are fully composable. They simply prune specific hyperedges, leaving a sub-hypergraph.
- Ranking and top-k are not natively composable because the hyperedges in our model are unordered. However, composability